

Memoria técnica de desarrollo de Bckroom

Daniel Díaz García

Enero 2026

Índice

1. Introducción	1
2. Historias de Usuario	2
3. Tecnologías Usadas	3
4. Gestión de Configuración	4
4.1. Control de Versiones	4
4.1.1. Estrategia de Ramas	4
4.1.2. Formato de Commits	4
4.2. Gestión de Versiones	5
4.3. Gestión de Actividades	5
5. Arquitectura	6

1. Introducción

Se pretende desarrollar un sistema similar a Airbnb donde personas particulares pueden reservar alojamientos de propietarios particulares. Tanto los inquilinos como propietarios deben registrarse previamente para utilizar el sistema.

- Los propietarios (una vez logeados) pueden dar de alta propiedades (viviendas completas o habitaciones individuales) ofertándolas para ser alquiladas. Los propietarios marcan si desean que los usuarios realicen reservas directas, o guardarse el derecho de confirmar o no las solicitudes de reserva.
- Los usuarios pueden primero buscar alojamiento (destino, fechas, tipo de inmueble, etc.) para examinar las opciones de alquiler. Excepcionalmente también pueden aplicar filtros avanzados para, por ejemplo: ver aquellos inmuebles con posibilidad de reserva inmediata; seleccionar filtros para ciertas comodidades; o seleccionar política de cancelación de reserva.
- Para realizar la búsqueda de inmuebles no es necesario estar logeado, pero los usuarios registrados pueden agregar algunas propiedades a su lista de deseo.
- Los inquilinos logeados, una vez han seleccionado el inmueble a alquilar, pueden completar la reserva. Si la propiedad permite reserva inmediata se puede completar el pago directamente (vía tarjeta de crédito, débito o paypal).
- Si la propiedad o inmueble no permite reserva inmediata se hace una solicitud de reserva (completando también en este momento el pago). La solicitud de reserva es notificada al propietario quién puede confirmar o no la reserva. Si la confirmación es positiva, esto se le notifica al inquilino. Si la solicitud es denegada, el sistema devuelve el dinero al inquilino y es igualmente notificado al cliente.

En este documento se describirá todo lo necesario para poder desarrollar la aplicación web llamada Bckroom. El documento contendrá las historias de usuario, arquitectura y diseños de la aplicación, un control de configuración y control de la calidad.

Para este proyecto se realizará con la metodología ágil SCRUM. Los sprints tendrán una duración de 3 semanas, las reuniones semanales se realizarán los viernes y las reuniones daily tendrán una duración máxima de 15 minutos. También se realizarán reuniones al finalizar cada sprint de una duración máxima de 2 horas y 30 minutos y mínima de 30 minutos en la que se analizará las funcionalidades implementadas, los problemas surgidos con las funcionalidades y comentar el retraso de las funcionalidades, si se producen.

2. Historias de Usuario

Las historias de usuario de la aplicación Bckroom son las siguientes. Las historias de usuario son la descripción de funcionalidades tal y como la percibe el usuario.

HU1: Como usuario sin iniciar sesión quiero poder buscar alojamientos para poder ver inmuebles sin cuenta.

- HU2: Como usuario del sistema quiero poder aplicar ciertos filtros para poder buscar alojamientos que se adapten a lo que necesito.
- HU3: Como usuario sin cuenta quiero poder crearme una cuenta de inquilino para poder guardar inmuebles a una lista de deseados o reservar alojamientos.
- HU4: Como usuario sin cuenta pero con propiedades quiero poder crearme una cuenta de propietario para poder ofertar para reserva las propiedades.
- HU5: Como usuario sin iniciar sesión quiero poder iniciar sesión con una cuenta de usuario para poder reservar alojamientos o poder ofertar alojamientos.
- HU6: Como inquilino quiero poder reservar inmuebles para poder disfrutar del alojamiento.
- HU7: Como inquilino quiero poder añadir inmuebles a una lista de deseos para poder guardar alojamientos que me gusten.
- HU8: Como inquilino quiero poder pagar vía tarjeta de crédito, débito o Paypal una reserva para poder completar la reserva de un inmueble.
- HU9: Como propietario quiero poder dar de alta propiedades para poder ofertarla para ser alquilada.
- HU10: Como propietario quiero poder indicar si se puede reservar inmediatamente o no un alojamiento para poder guardarse o no el derecho de aceptar o no las solicitudes.

3. Tecnologías Usadas

En esta sección se describirán las tecnologías utilizadas en el proyecto. Las tecnologías usadas son las siguientes:

- **Maven.** Para gestionar dependencias de manera sencilla.
- **Spring Web.** Para poder realizar una aplicación web.
- **Spring Security.** Para poder cifrar las contraseñas de los usuarios.
- **Spring JPA.** Para ayudar en operaciones de persistencia en base de datos relacional.
- **Apache Derby.** Como base de datos relacional embebida para no requerir de servidor ni instalación.

4. Gestión de Configuración

4.1. Control de Versiones

Para controlar el versionado del proyecto se va a utilizar la herramienta Git, y el repositorio remoto se va a encontrar alojado en Github.

4.1.1. Estrategia de Ramas

En este proyecto habrá dos ramas fijas que son main y development, main es la rama principal del proyecto, aquella que contendrá las versiones finales del sistema, y development esta creada desde main y esta rama contendrá el sistema en desarrollo. Después habrá varios tipos de ramas, que son temporales, que son las ramas feature, fix, hotfix y releases, todas estas ramas son ramas temporales creadas con una intención específica.

Los tipos de ramas feature son ramas que sirven para desarrollar nuevas funcionalidades del sistema, este tipo de ramas se desarrollan de la rama development, cuando se termina de realizar la funcionalidad se combina a la rama development y se elimina. El formato de nombre de la rama sería el siguiente *feature/<modulo>-<descripción corta>*, el módulo se refiere a que tipo de funcionalidad mayor se esta haciendo el cambio y la descripción corta es la acción específica que se realiza, por ejemplo, *feature/auth-login*.

Los tipos de rama fix sirven para poder realizar correcciones grandes del sistema, estas ramas se desarrollan de la rama de development, cuando se termina de realizar la corrección se combina a la rama development y se elimina. El formato de nombre de la rama sería el siguiente *fix/<modulo>-<descripción corta>*, por ejemplo, *fix/auth-login*.

Los tipos de rama release sirven para poder realizar pruebas sobre lo desarrollado antes de llevar los cambios a la rama main y cambiar documentación, estas ramas se desarrollan de la rama de development, cuando se termina de realizar las pruebas se combina a la rama main y se elimina. El formato de nombre de la rama sería el siguiente *fix/<vM.m.p>*, por ejemplo, *fix/1.5.2*.

Los tipos de rama hotfix sirven para poder realizar correcciones rápidas y pequeñas del sistema, estas ramas se desarrollan de la rama de main o release, cuando se termina de realizar la corrección se combina a la rama main y se elimina. El formato de nombre de la rama sería el siguiente *hotfix/<modulo>-<descripción corta>*, por ejemplo, *hotfix/auth-login*.

4.1.2. Formato de Commits

El formato de commits que se utilizará en este proyecto será el siguiente *<tipo>(<id de issue>): <mensaje corto en infinitivo>*. Los tipos de commits son los siguientes:

- **feat.** Indica nueva funcionalidad.
- **refactor.** Indica refactorización sin cambiar comportamiento.
- **fix.** Indica corrección de errores.
- **docs.** Indica documentación.
- **test.** Indica pruebas.
- **style.** Indica cambios en el estilo, formato, etc.
- **chore.** Indica tareas técnicas.

4.2. Gestión de Versiones

En este proyecto se va a usar el modo de versionado MAJOR.minor.patch, para cada sprint se verá cuantas funcionalidades o cambios aprobados se han alcanzado, se asignará el número de versión correspondiente y etiquetar con un commit concreto la versión del proyecto en la rama master.

En el repositorio habrá un archivo de changelog en donde se organizarán los cambios de cada versión estable. Este archivo se tendrá que editar y actualizar antes de realizar el commit final que contendrá el tag.

Después de que todos los cambios realizados hayan pasado el visto bueno y se haya actualizado el changelog y se realice la etiqueta del commit se creará una liberación, se le pone un nombre que sea vM.m.p correspondiente y la descripción será la parte del changelog correspondiente a esa versión.

4.3. Gestión de Actividades

Las actividades, o *issues* en el entorno de Github, deberán tener un título descriptivo y breve y un descripción explicativa y extensa, si es posible también insertar diagramas para facilitar el entendimiento del funcionamiento de la funcionalidad descrita. Las actividades pueden estar compuestas de subactividades, para poder detectarlas se estudiará si estas actividades pueden ser divididas, si se puede dividir se dividirá, y así hasta que no se pueda dividir más.

También se insertará una prioridad de la actividad conforme si esta se considera prioritaria para continuar con el desarrollo de la aplicación.

Luego la actividad también contendrá una estimación del tiempo en la que se va a llevar a cabo la tarea esta se calculará con la siguiente fórmula.

$$T_{Est} = \frac{T_{Pes} + 4 * T_{Pr} + T_{Opt}}{6} \quad (1)$$

Donde T_{Est} es el tiempo estimado, T_{Pes} es el tiempo más pesimista para realizar la actividad, T_{Opt} es el tiempo más optimista para realizar la actividad y T_{Pr} es el tiempo más probable que se prevé hacer la actividad. Conforme a la estimación se pondrá también cuando se ha empezado y acabará la actividad.

Por último, también se pondrá un tamaño de actividad, que indica la cantidad de trabajo que se requerirá para realizar una actividad.

5. Arquitectura

La arquitectura de la aplicación web va a ser Modelo-Vista-Controlador, basado en la re-utilización de código y separación de componentes, facilitando el desarrollo y mantenimiento de la aplicación. Con la tecnología que se va a utilizar para realizar la aplicación, que es Apache Maven, se utilizará otra capa extra, que es la capa de servicio, que será la encargada de contener la lógica más compleja utilizada en el controlador para hacer

escalable y mantenible el backend del sistema, ya que si se decide realizar una API o extensión de la funcionalidad futura en la re-utilización de lógica.